Which bits can we drop?

Component-wise quantization of a learned image codec

PULSE-MSE · 14.18M parameters · 8 quality settings · one weight file

-60.4%
model size, 0.94% quality cost

14.4 MB
smallest variant worth deploying

97.6%
of 4-bit damage repaired by QAT

The model is wildly unbalanced

so one uniform precision is the wrong instrument

Group	Parameters	Share	Why it matters
Encoder E0-E5	12.13M	85.6%	11.6 MB of the four big blocks
Entropy H0-H3	1.34M	9.4%	5.9% of weights, 73% of decode time
Decoder D0-D8	0.71M	5.0%	cheap in bytes, fragile in quality

Eight QPs ship in ONE weight file, so a per-QP precision choice would not be deployable.

Precision damage is not uniform either: some components are fine at 8 bits, some lose 38% at 8 bits.

So the question is an allocation problem: one precision per component, sized and constrained once.

Parameter counts from artifacts/component_inventory.json

How we measured it

69,233 measurements, one component at a time

7 formats

FP16, BF16, FP8-E4M3, FP8-E5M2, INT8, INT4, FP4-E2M1

19 components

every conv-bearing block, weights only, everything else FP32

8 QPs x 2 sets

Kodak screens, CLIC Professional confirms, 8 quality settings

Quality metric: rate-equivalent penalty, penalty = 100 x [(1 + dbpp/100) x 2^(-dPSNR/slope) - 1]

It folds a quality change and a bitrate change into one number. Zero means indistinguishable from FP32.

0.96-1.05

additivity ratio on Kodak: single-component costs sum correctly

10/10

unit tests pass on the fake-quantization core

scripts/pipeline.py (metric) · scripts/analyze_additivity.py (validation)

Some components simply refuse

13 of 19 are safe at 8 bits; three are not



Intolerant at INT8

H3 entropy step: 38.15%

H0 hyper encoder: 10.09%

D5 renderer MLP: 8.06%

Together 3.8% of parameters.

Cheap at INT8

E2/E3 encoder DCBs: 0.51 / 0.43%

E1/E4 encoder DCBs: 1.35 / 0.71%

D1 decoder block: 0.12%

results/analysis/component_precision_penalty_heatmap.png · worst-QP penalty per component and format

This is an allocation problem, so we solved it as one

mixed-integer linear program, HiGHS solver

Variables

x[component, precision] - one bit, chosen once and shared across all 8 QPs.

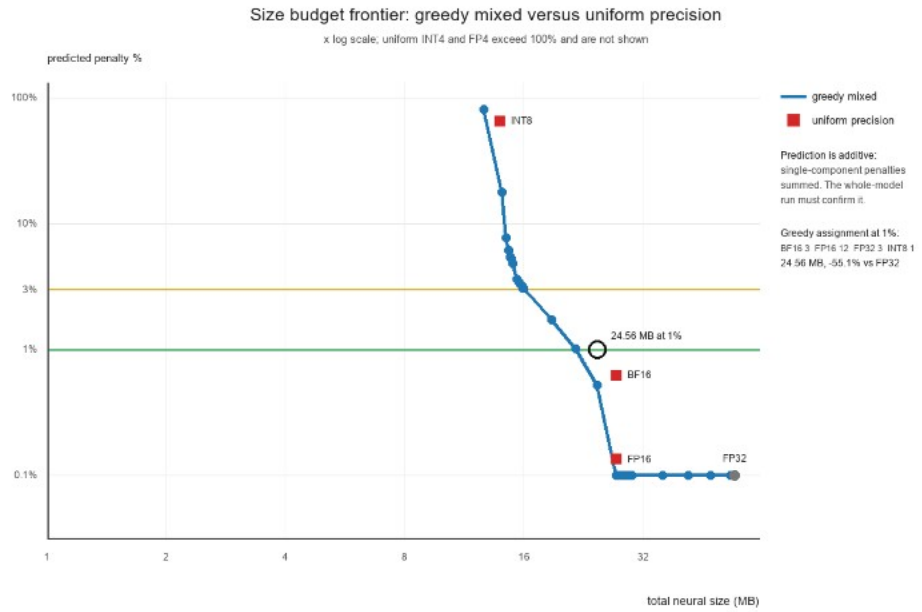
Constraint

For each QP q: sum of the measured component penalties at q <= budget.

Eight constraints, one per QP, because one weight file serves all eight.

Objective

Minimise total bytes. Not latency: we have no measured per-format latency table, so claiming one would be inventing data.



scripts/solve_ilp.py · scipy.optimize.milp (HiGHS) · 19 components x 7 formats

Three measured operating points

same harness, all three datasets, all eight QPs

Point	Size MB	Saving	Kodak	CLIC	Set14
1% budget champion	**21.677**	**-60.4%**	**0.67%**	**0.85%**	**0.94%**
1.8% budget variant	**18.664**	**-65.9%**	**1.27%**	**1.59%**	**1.60%**
INT8 tier + QAT	14.401	-73.7%	1.85%	2.87%	3.36%

FP32 is 54.688 MB. Every figure is a worst-QP rate-equivalent penalty against it.

1% champion: 0.94% worst case for 60% of the bytes gone.

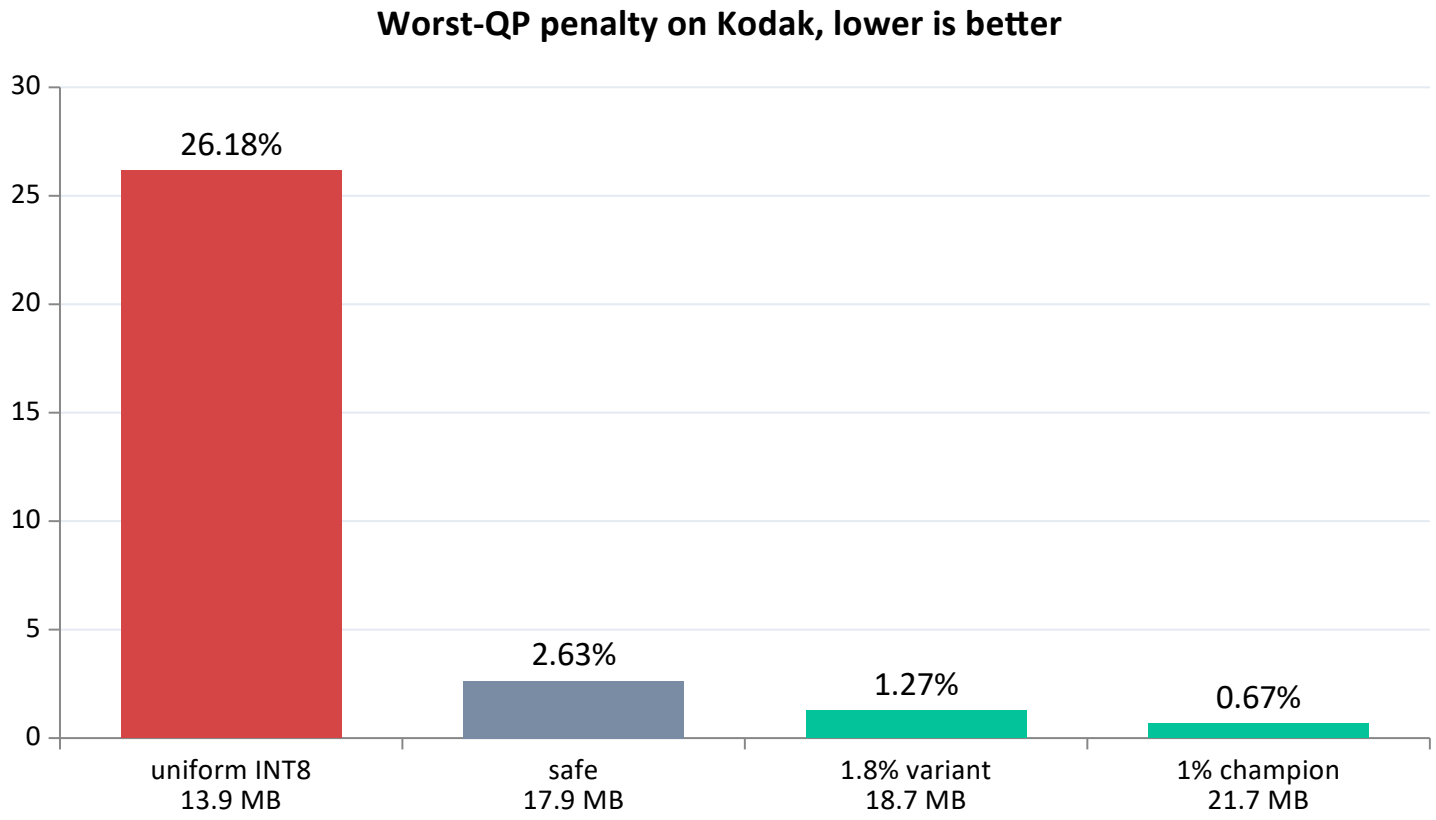
1.8% variant: 1.60% worst case for 66% gone.

INT8 tier + QAT: 3.36% worst case for 74% gone - and it only became usable because of fine-tuning.

results/final/SUBMISSION_POINTS.md

Mixed precision is the result

not a refinement: at comparable size, uniform INT8 is 39x worse



26.18%

uniform INT8, at 13.9 MB

0.67%

our 1% assignment, at 21.7 MB

The smaller model is 39x worse in quality. Spending bytes where sensitivity lives is worth more than spending them everywhere.

results/mixed/ comparison across 7 configs, 3 datasets

Where the size actually falls

the 1.8% budget replaced the 2% one

Budget	Size MB	Predicted %
1.0%	21.677	0.971
1.2%	21.396	1.137
1.4%	21.258	1.285
1.6%	21.151	1.444
1.8%	**18.664**	**1.716**
2.0%	18.389	1.986

Between 1.6% and 1.8% the size falls 2.5 MB in one step.

That is the knee: below it you pay quality for almost no bytes, above it you pay bytes for almost no quality.

Measured on all three datasets, the 1.8% assignment also beats the 2.0% one we had planned: 1.27 / 1.60 / 1.60% versus 1.40 / 1.94 / 1.71%.

So the earlier 2% choice was inside the knee and is not on the frontier. It was replaced.

results/night/ilp_fine_sweep.json · ilp_fine_measured.json

4-bit is not a knob you can just turn

even with group scaling it costs 26-157 points per MB

Comp	INT8 penalty	cost/MB	INT4g32 penalty	cost/MB
E3	0.427%	0.14	34.50%	31.4
E2	0.509%	0.17	28.85%	26.1
E4	0.710%	0.24	34.60%	31.2
E1	1.352%	0.46	172.27%	157.5
H3	38.152%	184	not viable	-
D5	8.057%	1712	not viable	-

cost/MB

penalty points paid per megabyte saved. The optimizer minimises exactly this.

Group-32 scaling cut 4-bit error by 40% and its damage 5-22x versus per-channel. Still about 100x worse per byte than the 8-bit step.

That is the honest 4-bit verdict: not worth it on its own, which is why we then tried to buy it back with fine-tuning.

results/qat/WHY_THESE_COMPONENTS.md · group-32 costs 5.0 effective bits per weight

Fine-tuning a quantized model, done carefully

three design choices that make it trustworthy

Constrained, not penalised

Minimise distortion subject to holding bitrate at its pre-quantization value. A fixed rate penalty either drifts or dumps quality to hit the target.

Entropy stays frozen

The bundle ships precomputed integer CDF tables. Changing entropy weights would break the bit-exact coding contract, so they are held fixed.

Best checkpoint wins

The saved model must beat its own starting point, or the starting weights are kept. A bad run cannot produce a worse model.

Augmented Lagrangian: loss = distortion + mu x (rate / target - 1), with mu updated from the measured rate error on a fixed validation set.

12,000 steps at 512-pixel crops, about 100 minutes per tier on one 4 GB laptop GPU.

That is 2.2% of the optimizer steps the released model was pretrained with, and 0.2% of its sample budget - fine-tuning, not retraining.

scripts/qat_run.py · scripts/qat_overnight.py

QAT recovers what the assignment leaves on the table

and nothing where it leaves nothing

Tier	Size MB	Before	2k steps	12k steps	Damage removed
More INT8	14.401	8.61%	8.61%	3.36%	61.0%
Moderate INT4	13.572	119.18%	25.27%	20.53%	82.8%
More INT4	11.369	1771.71%	84.22%	41.84%	97.6%

0.0%

QAT gain on the ILP 2% assignment (8,000 steps)

61 -> 98%

of damage removed across tiers at 12k steps

Three tiers with very different starting damage behave predictably: the worse the assignment, the more there is to recover.

But the ILP 2% model recovered exactly nothing, because a solver-optimised assignment has already spread its damage optimally. Fine-tuning has nothing to repair.

Correction we had to make: at 2,000 steps the INT8 tier gained 0.0%, which looked like 'QAT only helps severe damage'. At 12,000 steps it removed 61%. The first reading was a step-budget artefact.

results/final/qat_attempt/summary.csv · results/night/qat_int8_12k/summary.csv

We did not get a speedup, and we know exactly why

quantization accelerates the part that is not the bottleneck

Configuration	Latency	95% CI	Energy	Energy CI
ILP mixed vs FP32	+4.79%	+1.27 to +9.80	-0.39%	CI spans zero
FP16 autocast	+4.29%	-	-9.99%	-15.1 to -5.3
BF16 autocast	+1.82%	-	-8.86%	-

73%

of a GPU decode is entropy coding, not convolutions

9%

of a decode is the convolutional network we quantized

Simulated formats - FP8, INT4, FP4 - get no latency claim at all. We can only report size and quality for them.

A real INT8 speedup needs real INT8 kernels (TensorRT/ONNX), which is the obvious next engineering step, not a quantization-policy question.

Float formats are the only ones with a real kernel on this GPU, and they save energy rather than time.

results/metrics/ · paired interleaved benchmark, 50 pairs per cell, bootstrap CI

What we would ship

two lines of evidence, one recommendation

21.677 MB

champion - 60.4% smaller, 0.94% worst case

18.664 MB

smaller variant - 65.9% smaller, 1.60% worst case

Use the 1% assignment when quality is the constraint: it is inside 1% on all three datasets and its BD-rate is +0.48 to +0.55%.

Use the 1.8% assignment when size is the constraint: 3.0 MB smaller for about 0.7 extra points.

Do not use uniform INT8 at 13.9 MB: it is smaller and 39x worse. Do not use 4-bit without QAT: it is 21-42% even after 12,000 steps of repair.

Every number on this slide is measured on all three datasets at all eight QPs, with the FP32 reference gate passing.

results/final/SUBMISSION_POINTS.md

What would change our answer

four known limits, stated up front

Checkpoint selection under-reports

Our validation probe scored the best QAT model as no better than its start, while the full evaluation showed 8.61 to 3.36%. Selection uses 512-crops; the metric uses native resolution. Selection is leaving gains on the table.

Weights only

No activation quantization. W8A8 is the configuration that would actually use an INT8 kernel, and we did not build it.

Simulated 4-bit

FP8, INT4 and FP4 run as fake quantization. Quality numbers are real; any speed claim would not be.

One entropy lane

Decode spends 73% of its time in a single-lane rANS path. Multi-lane is the largest latency lever available and it changes the bitstream contract, so it is a design decision, not a tweak.

quantization-pluse-mse/STATUS.md section 4

One sentence

Mixed precision buys 60-66% of the model for under 2% quality; fine-tuning then repairs whatever the assignment could not afford - but it cannot improve an assignment that was already optimal.

21.677 MB at 0.94% · 18.664 MB at 1.60% · 14.401 MB at 3.36% after QAT